# Lab 05 — Web logs, unsafe input, and detection

**Course:** Maxim Nyansa Cybersecurity Bootcamp, Ghana
**Trainee:** James Ackah Blay · Aiyinasi, Ghana
**Week:** 5 — Application security and secure development
**Estimated time:** 5 hours
**Difficulty:** Intermediate. You will explain a web failure and detect it in a log. You will not collect a list of attacks.

> **Tutor note.** Authorisation: any local training application stays on `127.0.0.1` or on `192.168.56.0/24`. Do not point it at a network you do not own. This guide does not walk through exploiting an application. The required work is review, encoding, configuration, and detection.

## 1. Learning objectives

| Objective | Job skill |
|---|---|
| Place security work on the lifecycle, not only at the end | Talk to developers as a SOC or AppSec partner |
| Show that encoding changes how a browser treats text | Explain cross-site scripting impact and the fix |
| Review configuration without calling a hidden URL "secure" | Spot debug mode, default passwords, and secrets in git |
| Flag a suspicious web request in a log | Detection: what you would escalate, and what you would not |

## 2. Prerequisites and setup

Python 3 from the repository root, with `pytest` installed from `requirements.txt`. Read `../secure-code-review/` and `../xss-awareness/review-notes.md`.

Optional local application, only if Docker is available: OWASP Juice Shop, bound to localhost so it is not on the house LAN.

```
docker run --rm -d --name juice-shop -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

Open `http://127.0.0.1:3000` from the laptop. When you are finished:

```
docker stop juice-shop
```

If Docker is not installed, skip the container. The required detection work uses `samples/access.log` in this folder. Do not download a second vulnerable application "to try more things".

## 3. Scenario

A small training site your team owns has started recording odd requests. A developer asks whether "the firewall will handle it". Your job is to show where in the lifecycle the control belongs, to show a safe output function, and to say which log lines deserve a ticket.

## 4. Tasks

### Task 1 — Mark the lifecycle

Open `../secure-development-checklist/sdlc-checklist.csv`. Set a status only when you can point at work in this repository: `not-started`, `drafted`, or `done`.

### Task 2 — Encoding, not a scavenger hunt

From `../secure-code-review/`:

```
python3 -m pytest test_safe_input.py
python3 safe_input.py '<b>hello</b>'
```

**You should now see** the tests pass and the angle brackets encoded in the printed line. In your own words, write where this encoding has to happen: on the way out to HTML, not after you have already decided the text is “probably fine”.

Impact, in one sentence you should be able to say aloud: if a page builds HTML from text a user supplied, the browser may treat that text as a script in the reader’s session. The fix in this repo is `escape_html`. A Content-Security-Policy header is a second layer. It does not replace encoding.

Do not turn Juice Shop into a checklist of tricks. If you opened it, write what the front page is and confirm it is bound to `127.0.0.1`. Then stop.

## Task 3 — Configuration review

Fill `../insecure-configuration/config-checklist.csv` against Juice Shop only for settings you actually looked at, or leave the rows `not-reviewed` and say the container was not run. `not-present` is a valid result when you looked and did not see the issue.

## Task 4 — Detect in the log

`samples/access.log` is synthetic. It is not your server.

```
python3 web_log_review.py samples/access.log
```

The script flags requests that carry HTML markup in the URL. That is a detection for the lab, not a claim about every attack. Read the flagged lines. For each, write: what the client asked for, why it does not belong in a normal page view, and what you would ask the developer to change.

**You should now see** two flagged lines in the sample and at least one line that is not flagged. If you flag every line, the rule is too broad. Say so and tighten it only if you can explain the new rule in a sentence.

## Task 5 — Write the review row

In `../secure-code-review/review-report.md`, keep F-01 as the sample fix. Add F-02 only for something you saw: the sample log, the encoding test, or a setting you looked at. Status stays honest.

## 5. Troubleshooting

| What you see | What it means |
|---|---|
| Docker cannot bind port 3000 | Something else is listening. Stop it, or do the lab without the container |
| You can open Juice Shop from a phone on the house Wi-Fi | You published the port. Stop the container. Use `127.0.0.1:3000` as written |
| `pytest` cannot import `safe_input` | Run it from `secure-code-review`, or from the repo root after `pip install -r requirements.txt` |
| The detector flags a normal search | Your rule matched a character a real query might use. Record the false positive. Do not hide it |

## 6. Stretch task

Take one flagged sample line and write the detection the way you would hand it to a SIEM colleague: log source, the field you matched, one false positive, and the developer fix. Do not add further request strings.

## 7. Evidence

- pytest output for `safe_input`.
- The detector output on the sample log, labelled synthetic.
- F-02, or a sentence that says you have nothing further yet.
- If you ran Juice Shop: the `docker ps` line showing `127.0.0.1:3000`, then proof you stopped it.

## 8. Reflection

1. Why is encoding different from deleting the characters?
2. What is the harm if the log itself is the only control?
3. What did you refuse to do with the training application?

## 9. Interview questions

1. Where in the SDLC do you want input handling decided?
2. What is reflected cross-site scripting, in one sentence, and what is the first fix?
3. Why is a hidden admin URL not access control?
4. How do you avoid turning a detection rule into noise?

## 10. Grading rubric

| Criterion | Exceeds | Meets | Not yet |
|---|---|---|---|
| Lifecycle | Statuses point at real files | The checklist is touched honestly | Everything marked done |
| Encoding | You can explain the test and the impact | Tests pass and you kept the output | A list of strings to try in a browser |
| Detection | You separate flagged lines from normal lines | You ran the script and wrote what you would ask a developer | The sample log presented as a live incident |
| Scope | The container, if used, stayed on localhost | You skipped Docker and said so | The application was reachable on the home LAN |